

Relatório Técnico de Segurança Ofensiva e Defensiva

Laboratório de Rede Segmentada — VPN, Firewall, WAF e Cadeia de
Ataque

Processo Seletivo — Segurança da Informação
Documento confidencial — uso restrito à avaliação técnica

1. Sumário Executivo

Este relatório documenta a construção de um ambiente de rede corporativa segmentada e a execução de um ciclo completo de segurança sobre ele: reconhecimento, exploração, movimentação lateral, exfiltração de dados e, na sequência, a contenção de cada vetor identificado. O objetivo foi demonstrar, com evidência reproduzível, tanto a capacidade ofensiva (comprometer o ambiente) quanto a defensiva (segmentar, detectar e mitigar).

O ambiente reproduz, em um único host e com um único comando, um recorte representativo de uma infraestrutura híbrida: uma borda de VPN e firewall, uma DMZ com aplicação exposta atrás de um WAF, uma camada de aplicação interna e uma camada de banco de dados isolada. A premissa condutora foi o princípio de menor privilégio na rede: tudo o que não é explicitamente permitido é negado.

Principais resultados:

- Nove vulnerabilidades reais foram exploradas na aplicação-alvo, incluindo SQL Injection, execução remota de código via upload inseguro e injeção de comando.
- Uma cadeia de ataque de ponta a ponta foi comprovada: da aplicação exposta na DMZ até a exfiltração de dados sensíveis do banco, atravessando três segmentos de rede.
- A segmentação de rede demonstrou eficácia: mesmo com a credencial do banco em poder do atacante, o acesso direto ao banco a partir da DMZ permaneceu bloqueado.
- O WAF passou a bloquear os ataques (respostas 403) após a ativação do modo de prevenção; um bypass estrutural (client-side) foi identificado e mitigado com defesa em profundidade (Content-Security-Policy).
- Cada elo da cadeia recebeu mitigação, com comprovação de antes e depois.

2. Escopo e Metodologia

A avaliação seguiu a metodologia Cyber Kill Chain, percorrendo as fases de reconhecimento, armamento, entrega, exploração, instalação, comando e controle, e ações sobre o objetivo. O reconhecimento foi conduzido a partir de três pontos de vista distintos — externo (internet/host), interno via VPN e interno pós-comprometimento (DMZ) — de modo a mapear a superfície de ataque visível de cada posição.

A aplicação-alvo é o DVWA (Damn Vulnerable Web Application), executada atrás do WAF, conforme previsto no escopo do teste. As vulnerabilidades presentes na API interna (exposição de credencial e injeção de comando) são intencionais e documentadas, servindo para compor a cadeia de movimentação lateral.

3. Arquitetura da Infraestrutura

A infraestrutura é orquestrada com docker-compose em um único host, composta por quatro redes isoladas e um container-rodeador que concentra e filtra todo o tráfego entre segmentos. As três redes de segmento são internas (sem rota para a internet); o único ponto de contato externo é a porta da VPN, na rede de borda.

Segmento	Rede	Host	Papel
Borda	lab_edge	wg-gateway	Roteador/firewall (nftables) + entrada VPN (WireGuard)
DMZ	172.30.10.0/24	waf → dvwa	WAF (ModSecurity + CRS) à frente do alvo vulnerável
APP	172.30.20.0/24	app-api	API interna (pivô de movimentação lateral)
DB	172.30.30.0/24	postgres	Banco de dados com informações sensíveis

3.1. Segmentação e firewall default-deny

Redes Docker isoladas separam no nível de enlace, mas não oferecem controle por porta, registro de eventos nem rastreamento de estado de conexão. Por isso, o wg-gateway atua como roteador (multi-homed, conectado às três redes) executando nftables com política padrão de descarte (default-deny) nas cadeias INPUT e FORWARD. Apenas dois fluxos entre segmentos são liberados, de forma explícita e comentada:

- DMZ → APP, somente na porta da aplicação (8000).
- APP → DB, somente na porta do PostgreSQL (5432).

Qualquer outro caminho — por exemplo, DMZ → DB — recai no default-deny. O rastreamento de conexões (conntrack) garante que apenas o primeiro pacote de cada fluxo seja avaliado contra as regras; as respostas retornam pelo estado da conexão.

3.2. VPN e acesso administrativo restrito

O acesso administrativo (SSH) não é exposto à internet. O wg-gateway termina um túnel WireGuard, e o firewall aceita SSH somente quando a origem pertence à sub-rede da VPN. Como diferencial de hardening, o login administrativo é feito exclusivamente por

chave, com a autenticação por senha desabilitada — combinando controle de rede (somente via VPN) com controle de autenticação (somente por chave).

3.3. Serviços e WAF

Os serviços foram implantados nos segmentos corretos: o DVWA na DMZ, atrás do WAF; a API interna na camada APP; e o PostgreSQL na camada DB, fechado, sem exposição de porta ao host. À frente do DVWA, um Nginx com ModSecurity e o OWASP Core Rule Set atua como reverse proxy. O WAF inicia em modo de detecção (observa e registra sem bloquear) e, na fase defensiva, passa a operar em modo de prevenção (bloqueio com 403).

4. Reconhecimento e Enumeração

O reconhecimento foi conduzido a partir de três pontos de vista, mapeando o que está exposto de cada posição — e, igualmente importante, o que não está.

4.1. Ponto de vista externo (sem VPN)

A partir do host, sem a VPN ativa, a superfície do laboratório resume-se a uma única porta UDP (a da VPN, WireGuard). Nenhum serviço da DMZ, APP ou DB é enumerável externamente — a segmentação cumpre seu papel antes mesmo de qualquer autenticação.

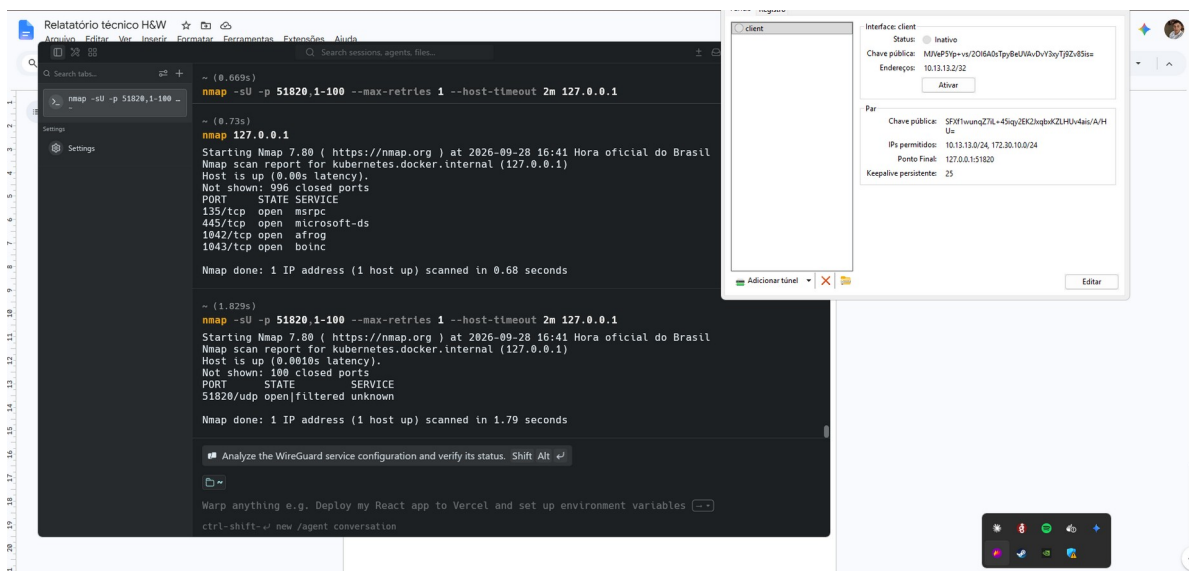


Figura 1 — Varredura externa sem VPN: superfície mínima, nenhum serviço do laboratório visível.

4.2. Ponto de vista interno via VPN

Com a VPN ativa, o único serviço alcançável passa a ser o WAF. O alvo (DVWA) não é acessível diretamente: todo o tráfego de entrada é obrigado a passar pelo WAF, conforme o desenho pretendido.

```
~ (0.551s)
nmap -Pn -sT -p 8080 172.30.10.5
Starting Nmap 7.80 ( https://nmap.org ) at 2026-09-28 16:59 Hora oficial do Brasil
Nmap scan report for 172.30.10.5
Host is up (0.0020s latency).

PORT      STATE SERVICE
8080/tcp  open  http-proxy

Nmap done: 1 IP address (1 host up) scanned in 0.51 seconds
```

Figura 2 — Com a VPN ativa, a aplicação torna-se alcançável através do WAF.

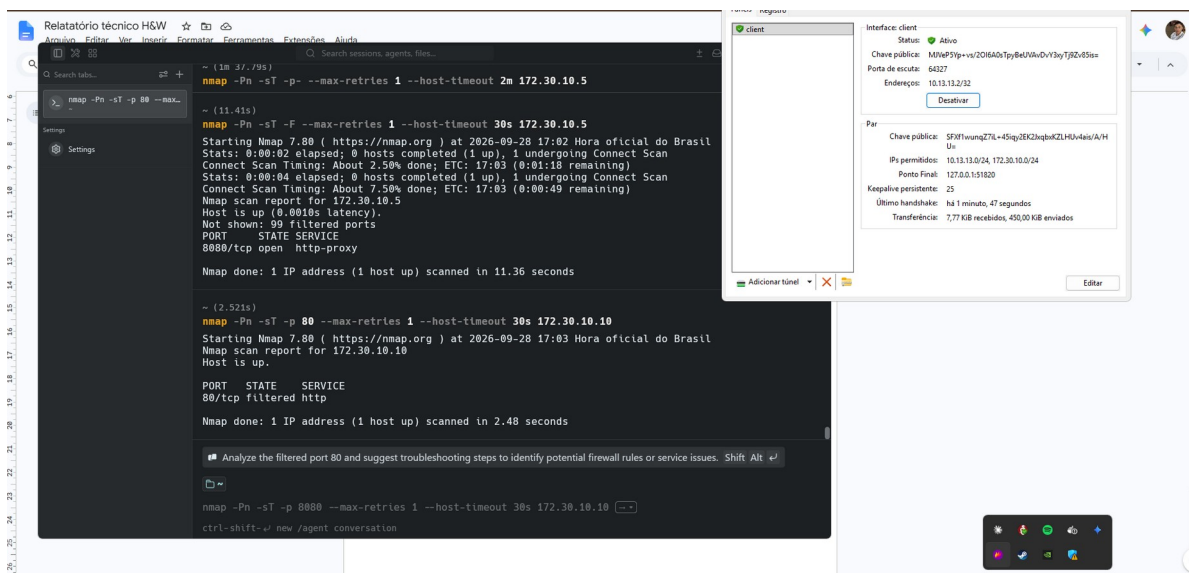


Figura 3 — A partir da VPN, apenas o WAF responde; o alvo não é acessível de forma direta.

4.3. Enumeração da aplicação

A identificação de tecnologia por meio dos cabeçalhos HTTP e dos cookies revelou o backend em PHP e expôs o nível de segurança da aplicação em um cookie legível — informação útil para o atacante. A enumeração de diretórios identificou arquivos sensíveis e o mapa completo dos módulos vulneráveis disponíveis para exploração.

```
~ (0.108s)
curl.exe -I http://172.30.10.5:8080/login.php
HTTP/1.1 200 OK
Date: Mon, 28 Sep 2026 20:10:56 GMT
Content-Type: text/html; charset=utf-8
Connection: keep-alive
Set-Cookie: PHPSESSID=r1cc0i538tofd7qsblv59lhli6; path=/
Expires: Tue, 23 Jun 2009 12:00:00 GMT
Cache-Control: no-cache, must-revalidate
Pragma: no-cache
Set-Cookie: PHPSESSID=r1cc0i538tofd7qsblv59lhli6; path=/
Set-Cookie: security=low
Server: nginx
Access-Control-Allow-Headers: *
```

Figura 4 — Identificação de tecnologia pelos cabeçalhos HTTP.

```
~ (0.057s)
curl.exe -s -o NUL -w "%{http_code} /robots.txt`n" http://172.30.10.5:8080/robots.txt
200 /robots.txt

~
curl.exe -s -o NUL -w "%{http_code} /README.md`n" http://172.30.10.5:8080/README.md
200 /README.md

~
curl.exe -s -o NUL -w "%{http_code} /config/config.inc.php`n" http://172.30.10.5:8080/config/config
.inc.php
200 /config/config.inc.php
```

Figura 5 — Cookies expondo o nível de segurança da aplicação.

```
~ (0.246s)
C:\Users\guilam\Documents\H-W-TechnicalTest\scripts\ffuf.exe -u http://172.30.10.5:8080/FUZZ -w C:\Users\guilam\Documents\H-W-TechnicalTest\scripts\wordlists\common-web.txt -mc all -t 20

      _____
     /  _  \  ____
    /  ___ \|  __|
   /  ___ \|  __|
  /  ___ \|  __|
 /  ___ \|  __|
/  ___ \|  __|
_____

v2.3.0

-----
:: Method          : GET
:: URL             : http://172.30.10.5:8080/FUZZ
:: Wordlist         : FUZZ: C:\Users\guilam\Documents\H-W-TechnicalTest\scripts\wordlists\common-web.txt
:: Follow redirects : false
:: Calibration      : false
:: Timeout          : 10
:: Threads          : 20
:: Matcher          : Response status: all

-----
config                [Status: 301, Size: 311, Words: 20, Lines: 10, Duration: 10ms]
favicon.ico           [Status: 200, Size: 1406, Words: 5, Lines: 2, Duration: 14ms]:
robots.txt            [Status: 200, Size: 26, Words: 3, Lines: 2, Duration: 16ms] ::
setup.php             [Status: 200, Size: 4077, Words: 308, Lines: 123, Duration: 19ms]
phpinfo.php           [Status: 302, Size: 0, Words: 1, Lines: 1, Duration: 19ms]0 ::
```

Figura 6 — Arquivos e diretórios de interesse identificados na enumeração.

5. Exploração e Cadeia de Ataque

Foram exploradas nove vulnerabilidades reais. Em seguida, os achados foram encadeados em uma cadeia de ataque de ponta a ponta, da aplicação exposta até a exfiltração do banco de dados.

5.1. SQL Injection

O campo de identificação de usuário concatena a entrada diretamente na consulta SQL, sem validação nem parametrização. Um payload de condição sempre verdadeira transforma a consulta e retorna todos os registros da tabela de usuários, em vez de um único. Impacto: exposição integral de dados de autenticação. Severidade: Crítica.

The screenshot displays the DVWA web application interface. On the left is a sidebar with navigation links: Home, Instructions, Setup / Reset DB, Brute Force, Command Injection, CSRF, File Inclusion, File Upload, Insecure CAPTCHA, SQL Injection (highlighted), SQL Injection (Blind), Weak Session IDs, XSS (DOM), XSS (Reflected), XSS (Stored), CSP Bypass, JavaScript, DVWA Security, PHP Info, About, and Logout. The main content area is titled 'Vulnerability: SQL Injection' and contains a 'User ID' input field with a 'Submit' button. Below the input, the results of the query are shown: 'ID: 1', 'First name: admin', and 'Surname: admin'. A 'More Information' section lists several links to external resources about SQL Injection. At the bottom right, there are 'View Source' and 'View Help' buttons. The footer of the application states 'Damn Vulnerable Web Application (DVWA) v1.10 *Development*'.

Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript
DVWA Security
PHP Info
About
Logout

Vulnerability: SQL Injection

User ID:

ID: 1
First name: admin
Surname: admin

More Information

- <http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
- https://en.wikipedia.org/wiki/SQL_injection
- <http://ferruh.mavituna.com/sql-injection-cheatsheet-oku/>
- <http://pentestmonkey.net/cheat-sheet/sql-injection/mysql-sql-injection-cheat-sheet>
- https://www.owasp.org/index.php/SQL_injection
- <http://bobby-tables.com/>

View Source View Help

Username: admin
Security Level: low
PHPIDS: disabled

Damn Vulnerable Web Application (DVWA) v1.10 *Development*

Figura 7 — SQL Injection: a consulta manipulada retorna todos os usuários.



Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript

DVWA Security

PHP Info

About

Logout

Vulnerability: SQL Injection

User ID:

Submit

ID: 1' OR '1'='1
First name: admin
Surname: admin

ID: 1' OR '1'='1
First name: Gordon
Surname: Brown

ID: 1' OR '1'='1
First name: Hack
Surname: Me

ID: 1' OR '1'='1
First name: Pablo
Surname: Picasso

ID: 1' OR '1'='1
First name: Bob
Surname: Smith

More Information

- <http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
- https://en.wikipedia.org/wiki/SQL_injection
- <http://ferruh.mavituna.com/sql-injection-cheatsheet-okul/>
- <http://pentestmonkey.net/cheat-sheet/sql-injection/mysql-sql-injection-cheat-sheet>
- https://www.owasp.org/index.php/SQL_injection
- <http://bobby-tables.com/>

Username: admin
Security Level: low
PHPIDS: disabled

View Source

View Help

Damn Vulnerable Web Application (DVWA) v1.10 *Development*

Figura 8 — SQL Injection: dados de autenticação expostos.

5.2. SQL Injection Blind (baseado em tempo)

Quando a aplicação não exibe o resultado da consulta, a injeção ainda é comprovável pelo tempo de resposta. Um payload com função de atraso condicional faz a resposta demorar cinco segundos ou mais quando a condição é verdadeira, enquanto um payload de controle responde imediatamente — confirmando a execução do SQL injetado sem retorno visível de dados. Severidade: Crítica.

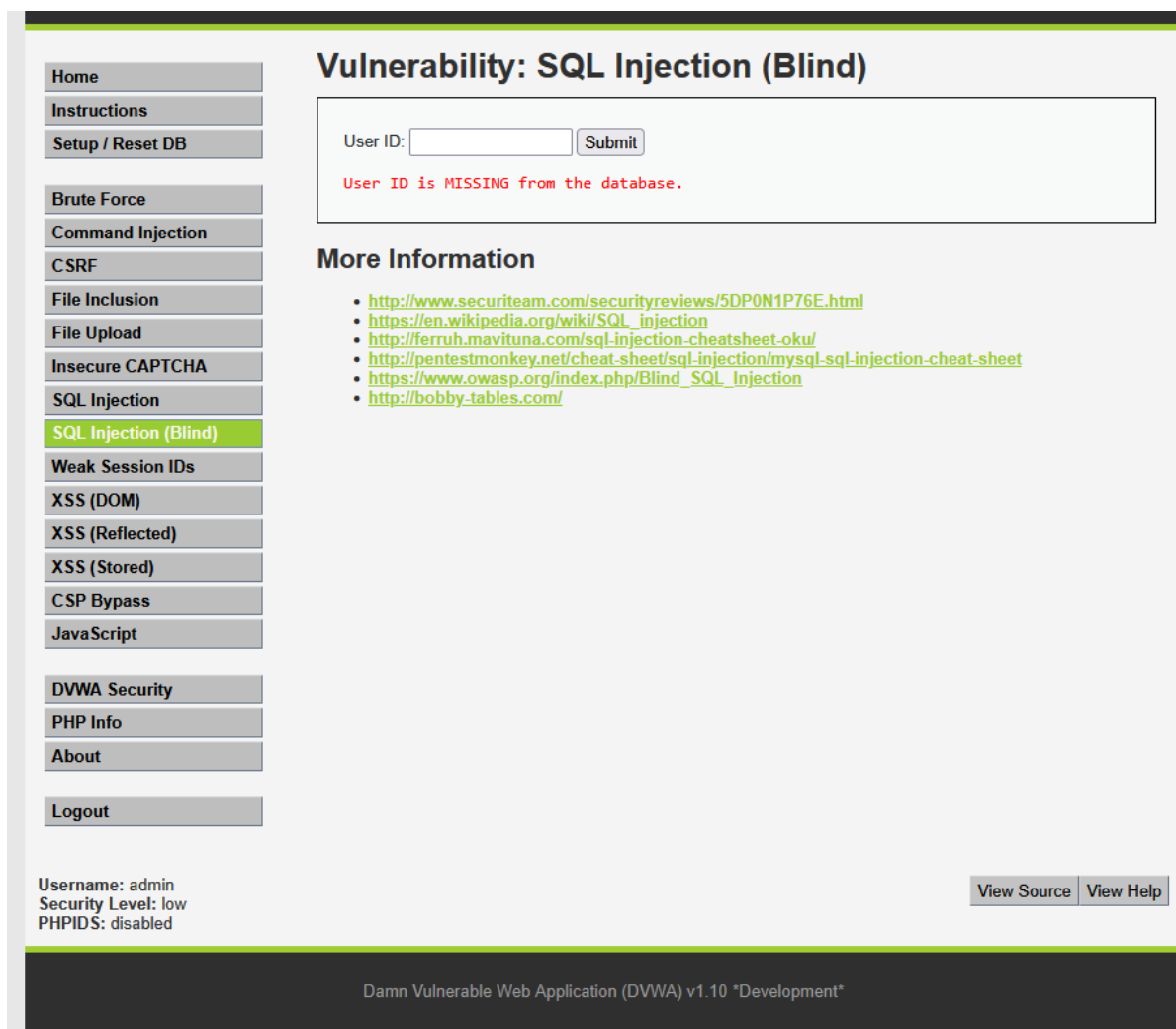


Figura 9 — SQL Injection Blind: atraso condicional confirma a injeção.

5.3. Cross-Site Scripting (XSS)

Foram identificadas três variantes de XSS. O XSS Refletido injeta script que é devolvido e executado na resposta imediata; embora dependa de o usuário acessar um link preparado, é uma vulnerabilidade real, com potencial de roubo de sessão e phishing. O XSS Armazenado é mais grave: o script é persistido pela aplicação e executado no navegador de todos os usuários que acessarem a página. O XSS baseado em DOM ocorre inteiramente no cliente — o servidor não chega a processar o payload — e será determinante na análise dos limites do WAF (Seção 6).

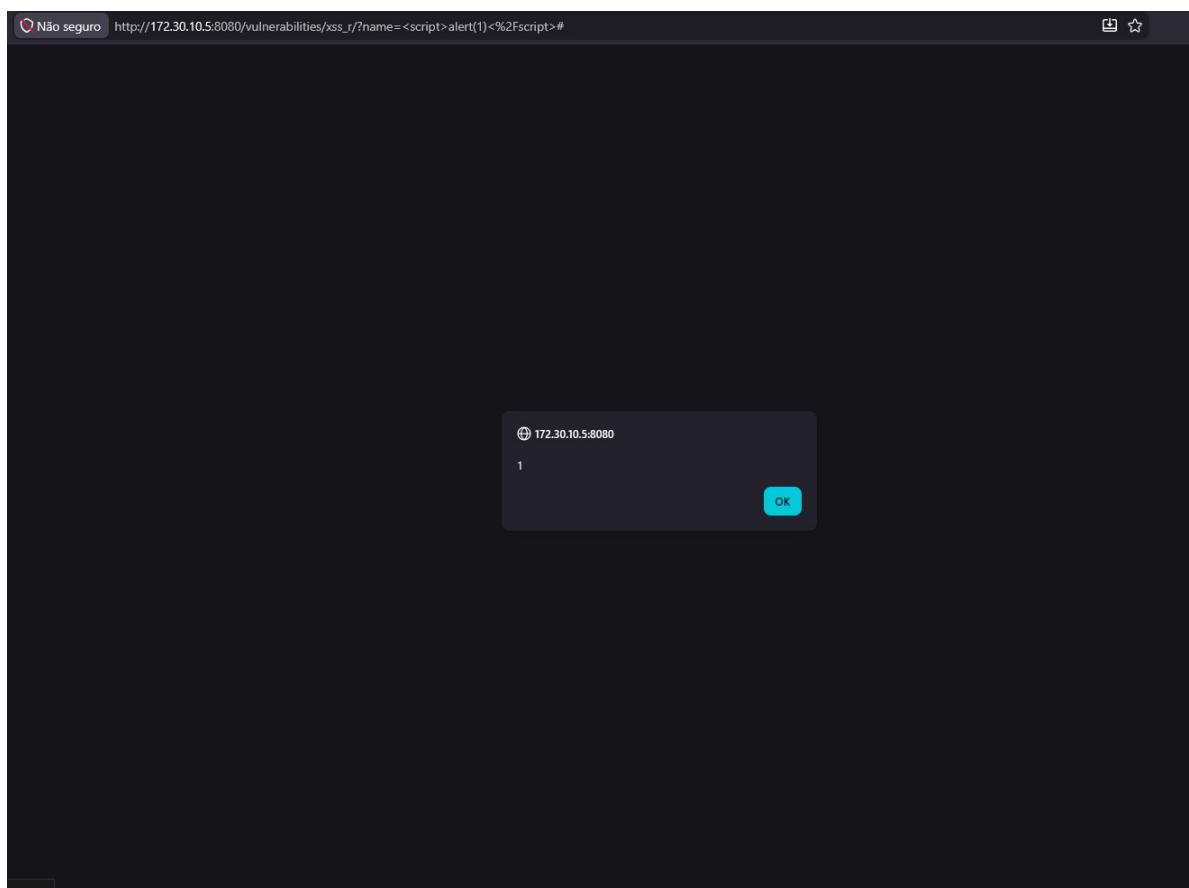


Figura 10 — XSS Refletido.

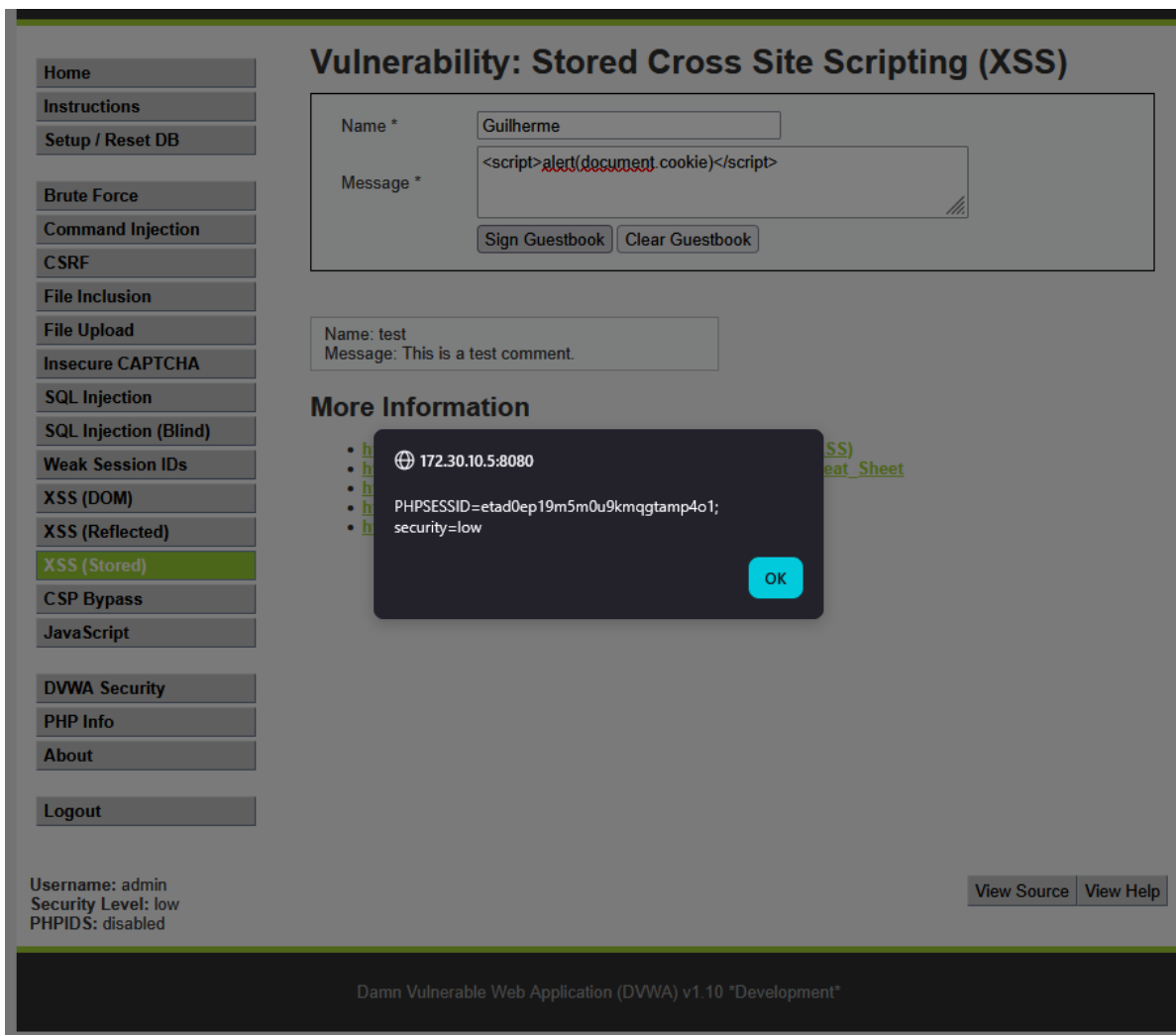


Figura 11 — XSS Armazenado (persistido pela aplicação).

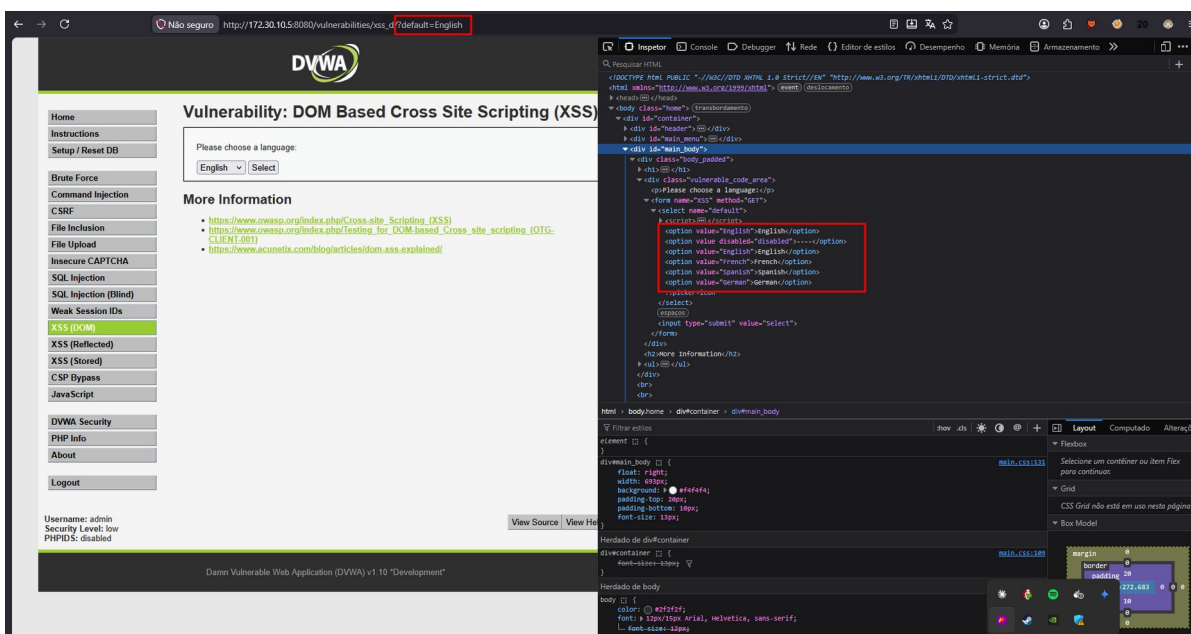


Figura 12 — XSS baseado em DOM (executado no cliente).

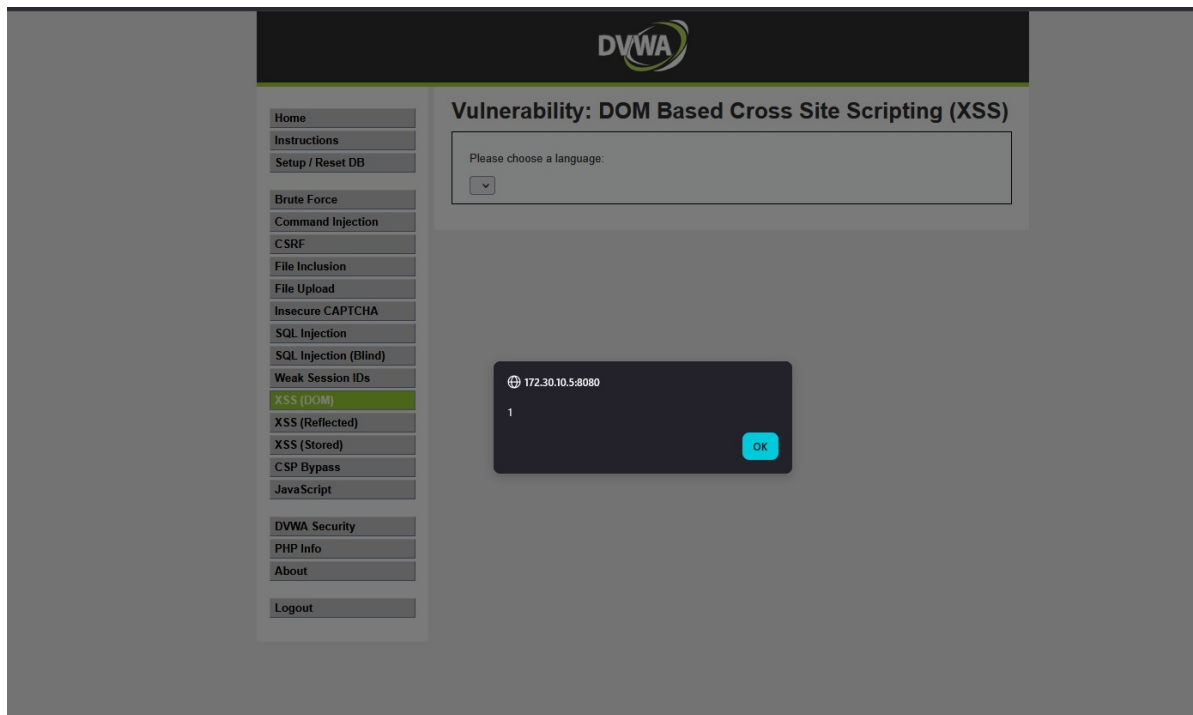


Figura 13 — Construção do payload de XSS DOM, encerrando as tags para injetar o script.

5.4. Upload inseguro e execução remota de código

O módulo de upload não valida extensão, tipo MIME nem assinatura do arquivo. Foi enviado um webshell em PHP, que passou a residir no diretório de uploads e, ao ser acessado, permitiu a execução de comandos no servidor. A confirmação do usuário do sistema (www-data) caracteriza execução remota de código e estabelece o foothold na DMZ. Severidade: Crítica.

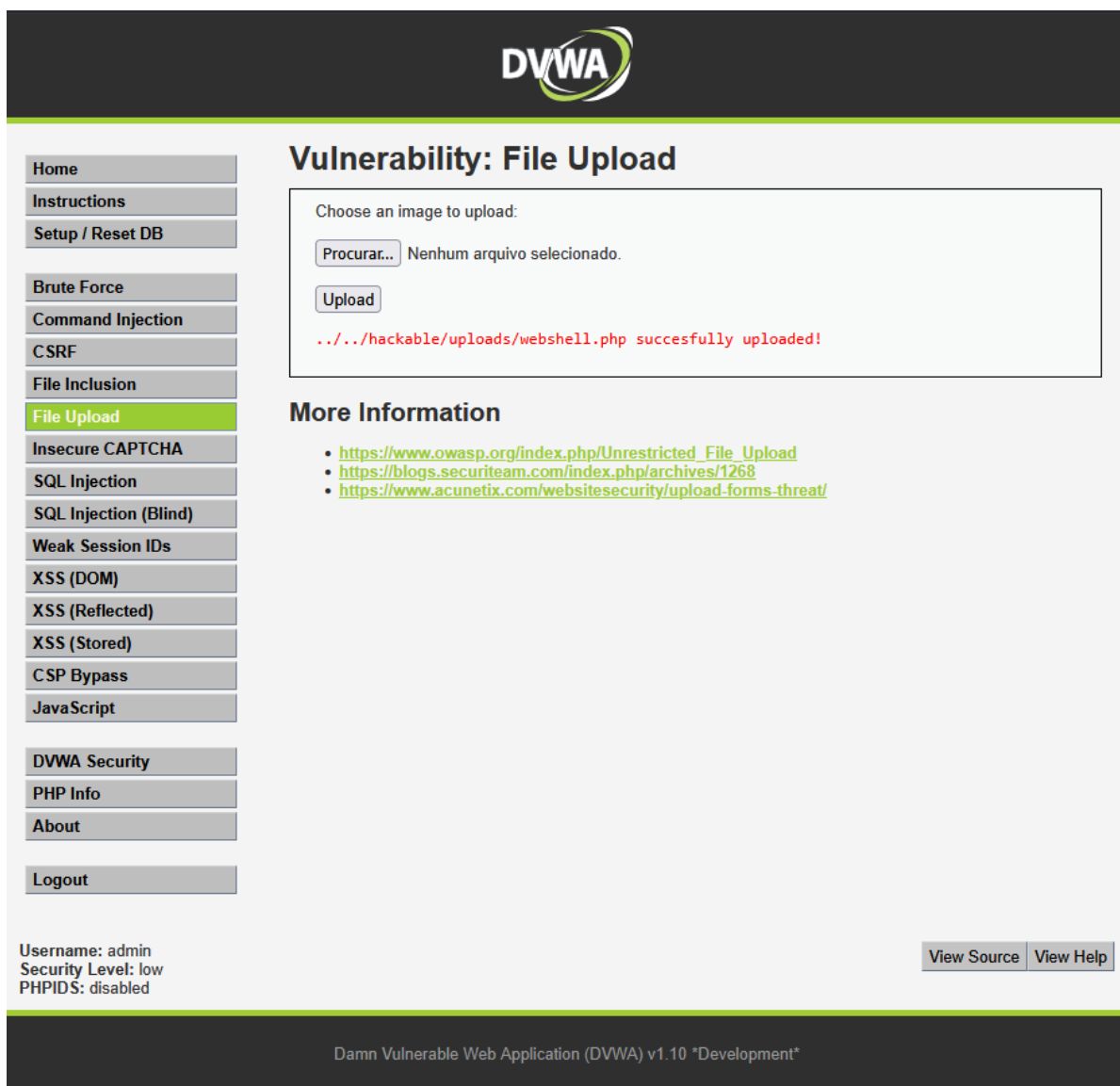


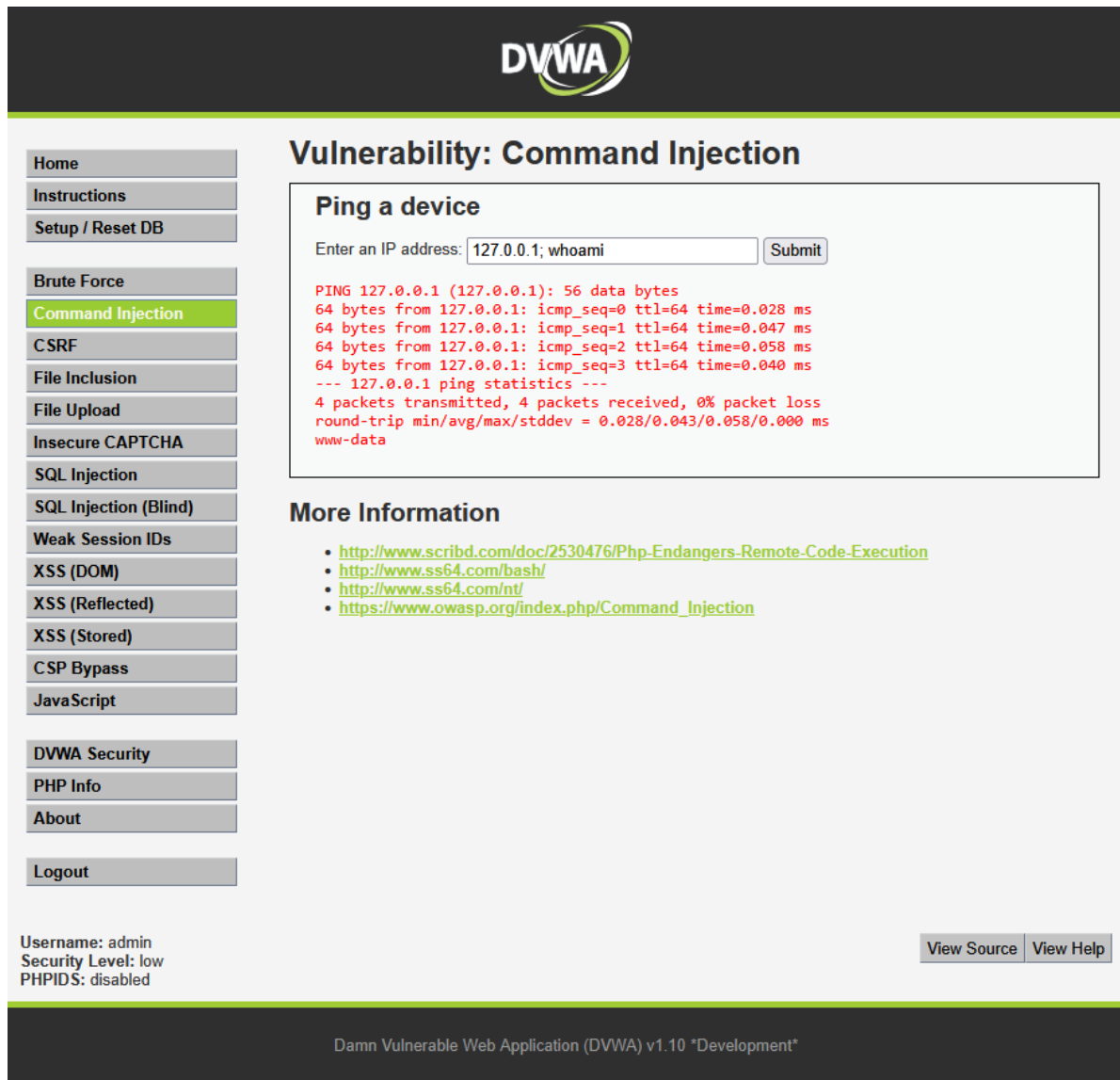
Figura 14 — Upload do arquivo malicioso aceito sem validação.



Figura 15 — Webshell ativa: execução de comando confirmada (foothold na DMZ).

5.5. Command Injection

Uma funcionalidade que executa um comando de sistema com parâmetro fornecido pelo usuário, sem sanitização, permite encadear comandos adicionais por meio de separadores de shell. O resultado é execução arbitrária de comandos além da função pretendida. Severidade: Crítica.



The screenshot displays the DVWA web application interface. The sidebar on the left contains navigation links: Home, Instructions, Setup / Reset DB, Brute Force, **Command Injection** (highlighted), CSRF, File Inclusion, File Upload, Insecure CAPTCHA, SQL Injection, SQL Injection (Blind), Weak Session IDs, XSS (DOM), XSS (Reflected), XSS (Stored), CSP Bypass, JavaScript, DVWA Security, PHP Info, About, and Logout. The main content area is titled 'Vulnerability: Command Injection' and features a 'Ping a device' section. This section includes a form with the label 'Enter an IP address:' and a text input field containing '127.0.0.1; whoami', followed by a 'Submit' button. Below the form, the output of the command injection is displayed in red text: 'PING 127.0.0.1 (127.0.0.1): 56 data bytes', '64 bytes from 127.0.0.1: icmp_seq=0 ttl=64 time=0.028 ms', '64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.047 ms', '64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.058 ms', '64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.040 ms', '--- 127.0.0.1 ping statistics ---', '4 packets transmitted, 4 packets received, 0% packet loss', 'round-trip min/avg/max/stddev = 0.028/0.043/0.058/0.000 ms', and 'www-data'. Below the output, a 'More Information' section lists four links: <http://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution>, <http://www.ss64.com/bash/>, <http://www.ss64.com/nt/>, and https://www.owasp.org/index.php/Command_Injection. At the bottom of the main content area, there are 'View Source' and 'View Help' buttons. The footer of the application shows the username 'admin', security level 'low', and PHPIDS status 'disabled'. The footer also includes the text 'Damn Vulnerable Web Application (DVWA) v1.10 *Development*'.

Figura 16 — Command Injection: execução de comando adicional além do previsto.

5.6. File Inclusion (LFI)

O parâmetro que determina qual arquivo será carregado não é validado, permitindo, por travessia de diretórios, a leitura de arquivos sensíveis do sistema — como o `/etc/passwd`. Essa exposição de arquivos locais pode revelar informações úteis para o aprofundamento do ataque. Severidade: Alta.

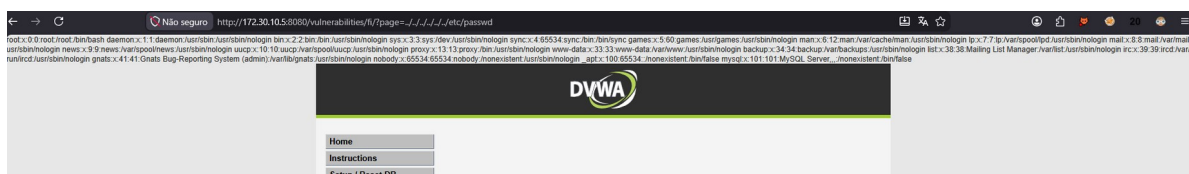


Figura 17 — Local File Inclusion: leitura de /etc/passwd por travessia de diretórios.

5.7. CSRF (Cross-Site Request Forgery)

Este vetor não explora a falta de validação de entrada, mas a confiança do navegador na sessão autenticada. Sem um token anti-CSRF, uma requisição de troca de senha pode ser disparada a partir de uma página externa: ao clicar em um link aparentemente inofensivo, o próprio navegador da vítima envia o cookie de sessão, e a senha é alterada sem a ação consciente do usuário. Nota técnica: navegadores modernos aplicam a proteção SameSite por padrão, que bloqueia o vetor via requisição embutida (por exemplo, uma tag de imagem), mas não a navegação de topo (clique em link) — o que foi utilizado para concluir o ataque. Severidade: Alta.

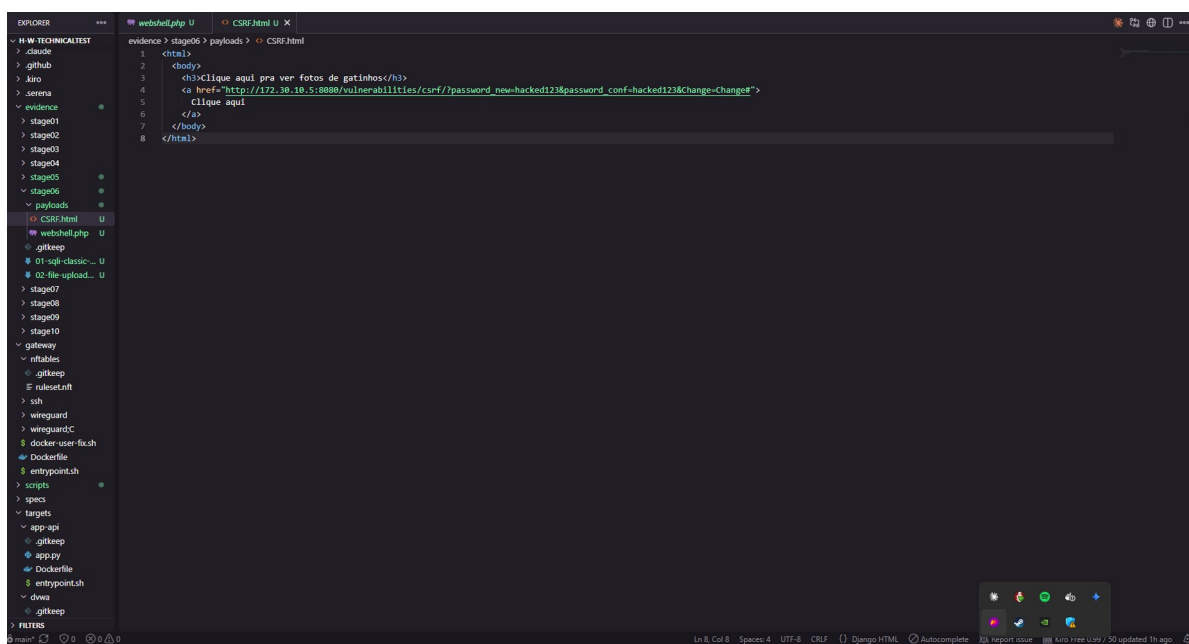


Figura 18 — Página de ataque CSRF preparada.



Figura 19 — Execução do CSRF por meio de navegação de topo.

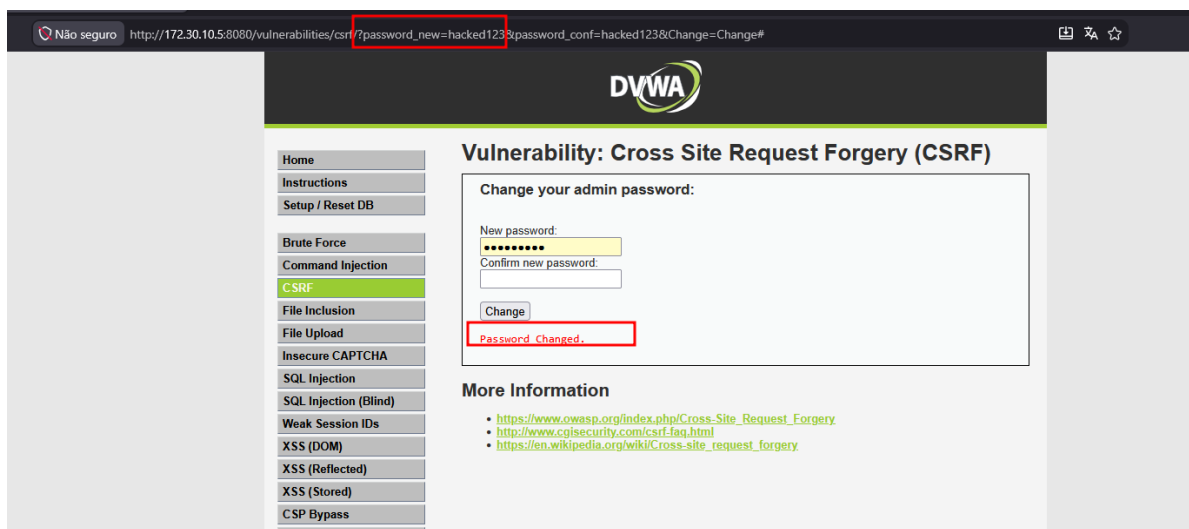


Figura 20 — Resultado: alteração de senha sem interação legítima do usuário.

5.8. Brute Force e identificador de sessão fraco

A ausência de limitação de tentativas (rate limiting) permite testar exaustivamente combinações de usuário e senha até obter acesso. Com o uso de listas de credenciais comuns, foram identificadas credenciais válidas. Adicionalmente, o gerador de identificadores de sessão produz valores estritamente sequenciais (incremento de um a cada chamada) — um identificador previsível que permitiria antecipar sessões de outros usuários sem qualquer roubo de credencial. Severidade: Alta.

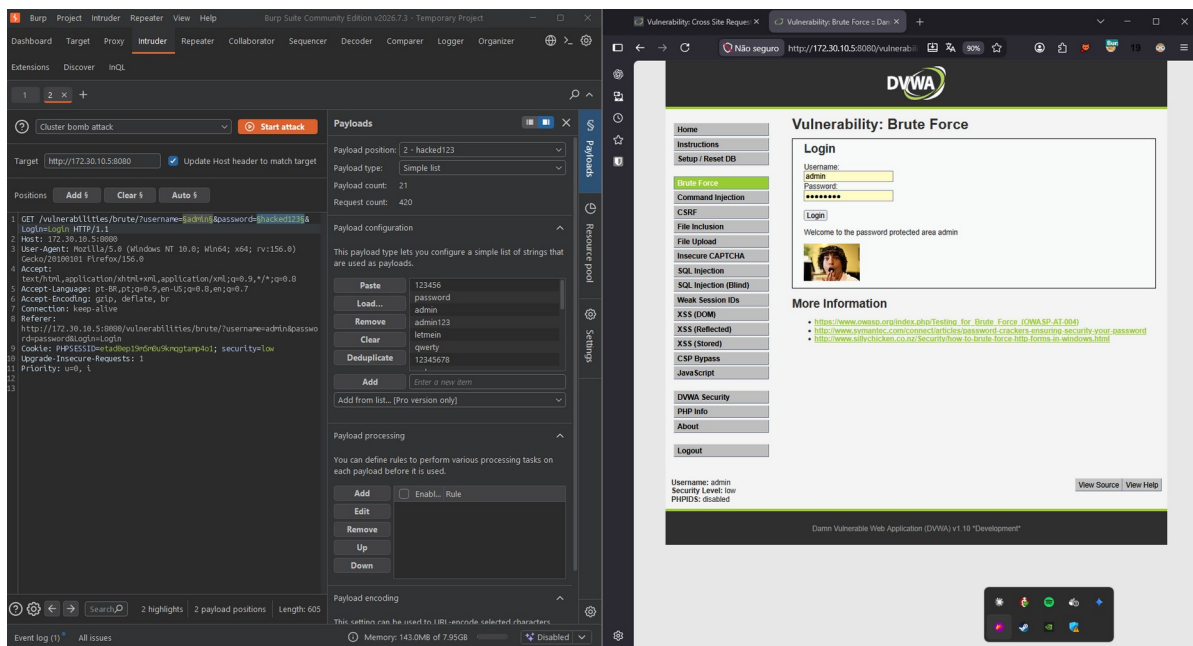


Figura 21 — Brute Force: ausência de limitação de tentativas.

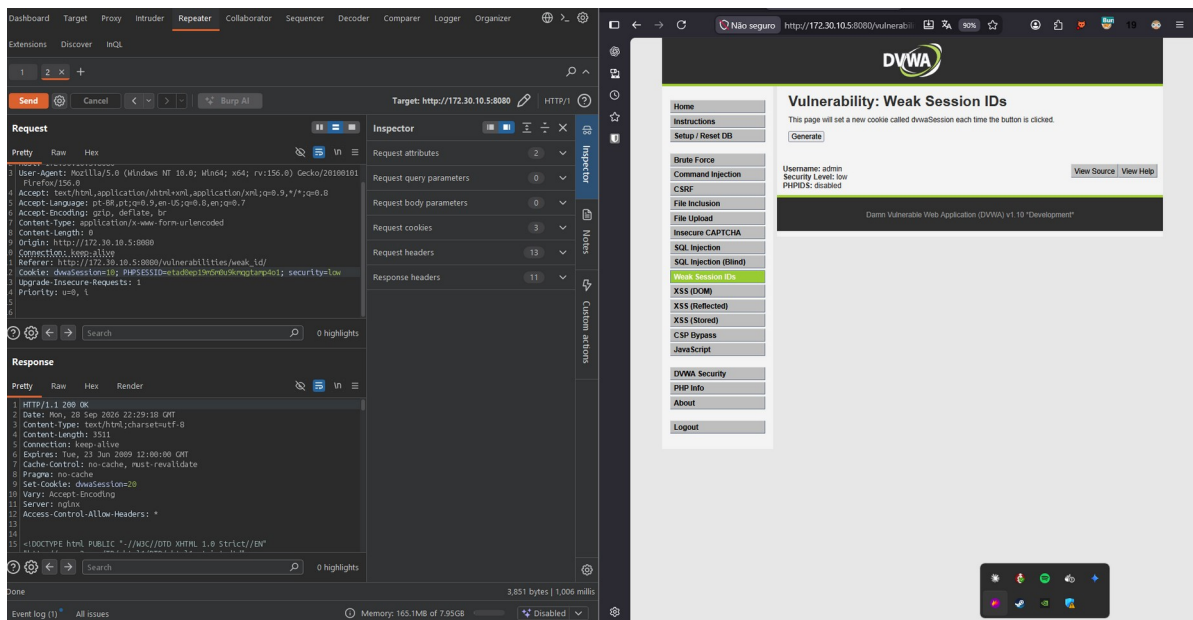


Figura 22 — Credenciais válidas identificadas.

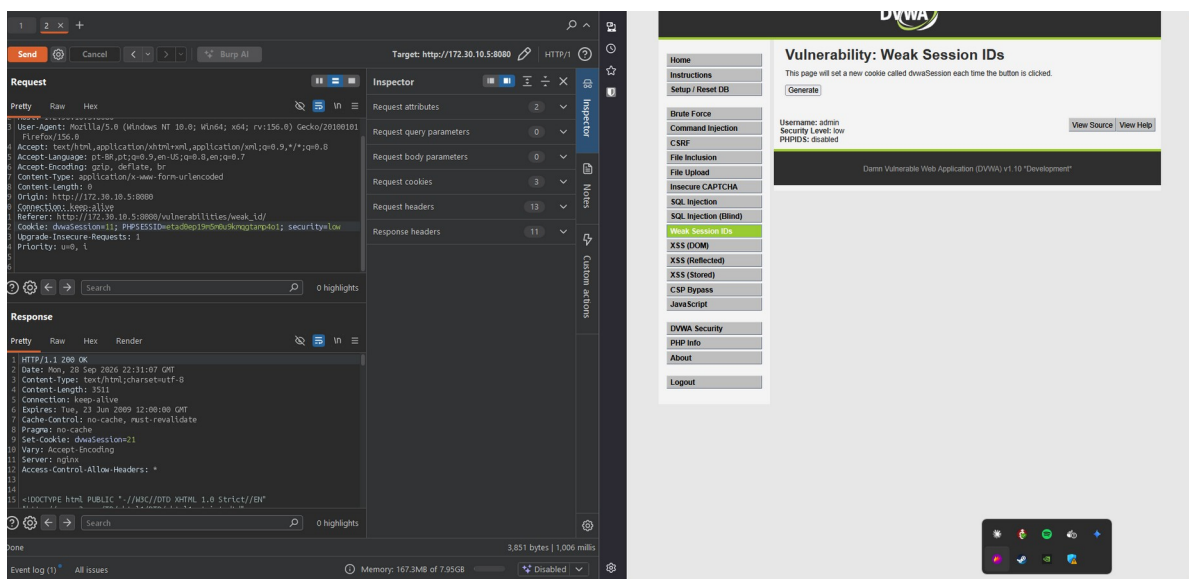


Figura 23 — Requisição do atacante durante o ataque.

5.9. Cadeia de ataque completa — da DMZ ao banco de dados

Com o foothold estabelecido na DMZ (webshell no DVWA), o ataque avançou para as camadas internas, respeitando — e evidenciando — a segmentação de rede:

- Movimentação lateral: a partir do webshell (origem na DMZ), a API interna na camada APP tornou-se alcançável, pois a regra de firewall permite o fluxo DMZ → APP.
- Exposição de credencial: um endpoint interno sem autenticação revelou a credencial de acesso ao banco de dados.
- Contenção pela segmentação: mesmo de posse da credencial, o acesso direto ao banco a partir da DMZ permaneceu bloqueado — a regra de firewall só permite o fluxo APP → DB. A credencial, isoladamente, não foi suficiente.
- Reúso de credencial via pivô: a partir de execução de comando na camada APP, a credencial roubada foi utilizada com o cliente de banco de dados para alcançar o PostgreSQL (fluxo legítimo APP → DB) e exfiltrar os dados sensíveis.

Essa distinção é relevante para a avaliação de risco: a defesa de rede funcionou — neutralizou o uso direto da credencial — e a exfiltração só foi possível através da cadeia legítima, explorando falhas de aplicação. É precisamente essa brecha de aplicação que a fase de contenção (Seção 7) endereça.



Figura 24 — Aprofundamento do ataque: alcance à camada de aplicação.

```
~  
curl.exe "http://172.30.10.5:8080/hackable/uploads/webshell.php?cmd=$encoded"  
<pre>string(114) "{"db_host":"172.30.10.10","db_name":"labdb","db_password":"S3nh4_D0_Lab_2026","db_port":5432,"db_user":"appuser"}"  
"
```

Figura 25 — Exfiltração dos dados sensíveis do banco de dados.

6. Defesa Reativa — WAF (Bloqueio, Bypass e Correção)

Concluída a fase ofensiva, o WAF foi migrado do modo de detecção para o modo de prevenção. As mesmas requisições que antes eram apenas registradas passaram a ser bloqueadas com resposta 403.

```
~ (0.182s)  
try { (Invoke-WebRequest "http://172.30.10.5:8080/?id=1%20UNION%20SELECT%20user%20FROM%20users").StatusCode } catch { "BLOQUEADO -> " + $_.Exception.Response.StatusCode  
.value__ }  
BLOQUEADO -> 403
```

Figura 26 — WAF em modo de prevenção: SQL Injection bloqueado (403).

6.1. Tentativas de bypass

A evasão por dupla codificação foi testada e não obteve sucesso: o Core Rule Set aplica funções de transformação que decodificam a entrada antes da análise, desfazendo a codificação e detectando o ataque. Isso demonstra a robustez do controle contra técnicas básicas de evasão.

```
~  
try { (Invoke-WebRequest "http://172.30.10.5:8080/?id=1%20UNION%20SELECT%20user").StatusCode } catch { "baseline BLOQUEADO -> " + $_.Exception.Response.StatusCode.value__ }  
baseline BLOQUEADO -> 403  
  
~  
# codificação dupla - o teste  
  
~  
try { (Invoke-WebRequest "http://172.30.10.5:8080/?id=1%2520UNION%2520SELECT%2520user").StatusCode } catch { "dobro BLOQUEADO -> " + $_.Exception.Response.StatusCode.value__ }  
dobro BLOQUEADO -> 403
```

Figura 27 — Tentativa de bypass por dupla codificação, bloqueada pelo WAF.

O bloqueio de XSS também foi confirmado. Em seguida, um bypass estrutural foi identificado: o XSS baseado em DOM, quando entregue pelo fragmento da URL (após o caractere '#'), não trafega até o servidor. Como o WAF inspeciona requisições, ele é estruturalmente incapaz de bloquear um ataque que ocorre inteiramente no cliente — não existe regra que feche essa lacuna, pois o payload nunca chega ao WAF.

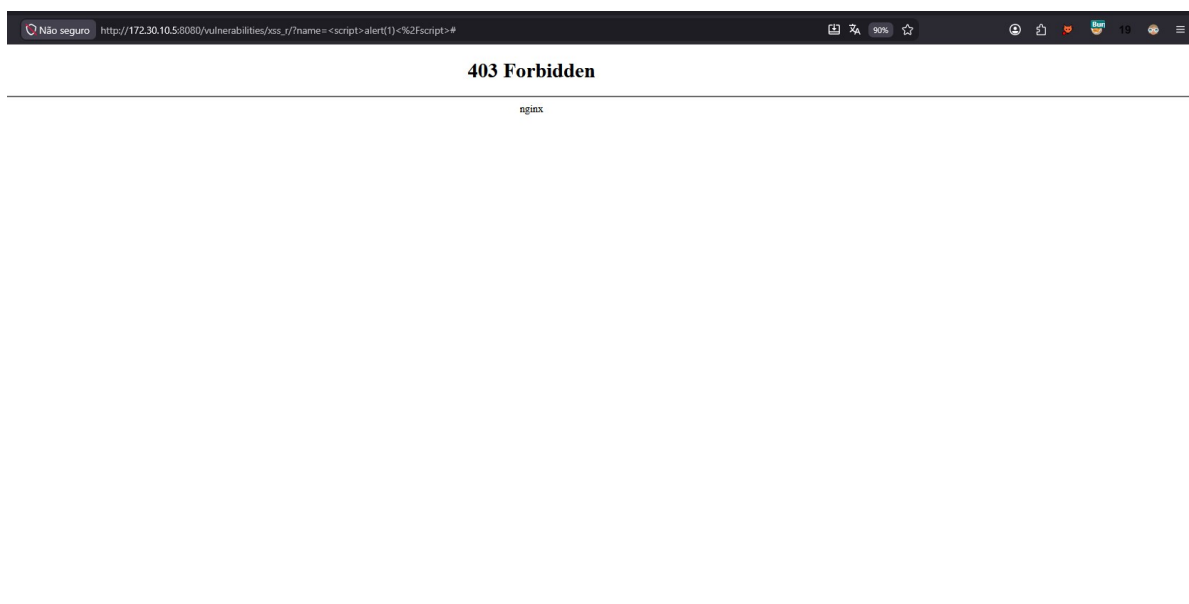


Figura 28 — Teste de bloqueio de XSS pelo WAF.

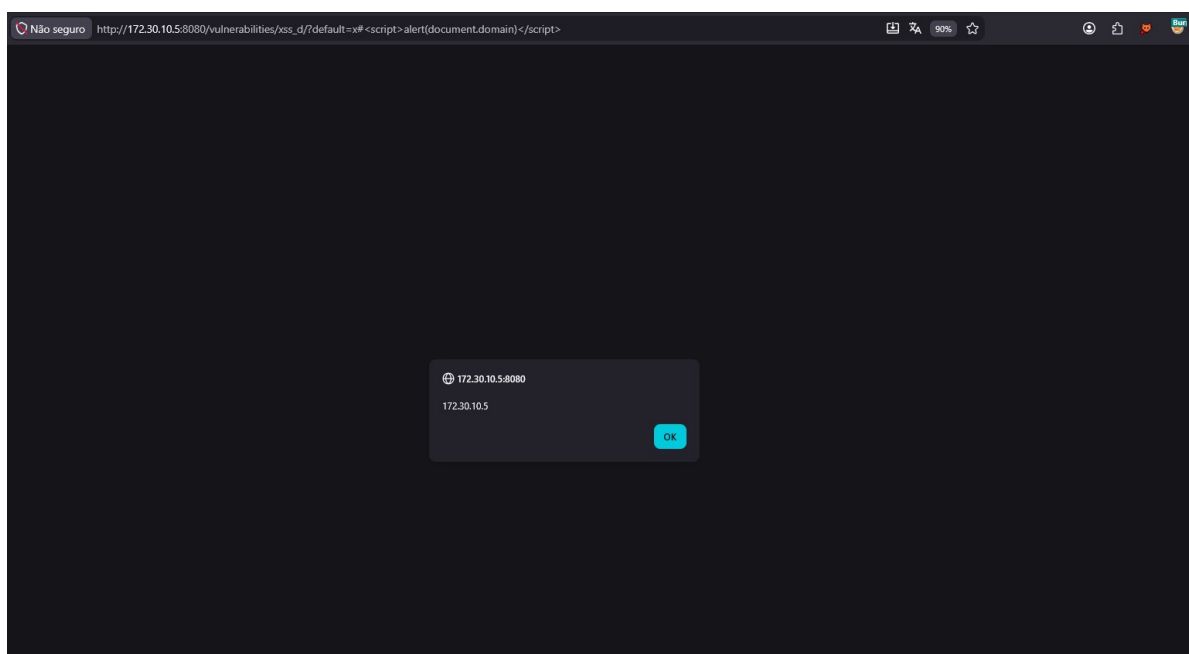


Figura 29 — Bloqueio de XSS confirmado e análise do bypass via fragmento.

6.2. Correção — defesa em profundidade com CSP

Uma vez que o WAF não pode inspecionar o fragmento, a correção adequada é de outra natureza: o WAF passou a injetar, na resposta, o cabeçalho Content-Security-Policy. Ele não depende de enxergar o payload — instrui o navegador a recusar a execução de scripts inline, neutralizando o XSS baseado em DOM no cliente. Confirmou-se que requisições legítimas continuam a ser atendidas (resposta 200, sem falso positivo) e que as proteções anteriores contra SQL Injection e XSS permanecem ativas.

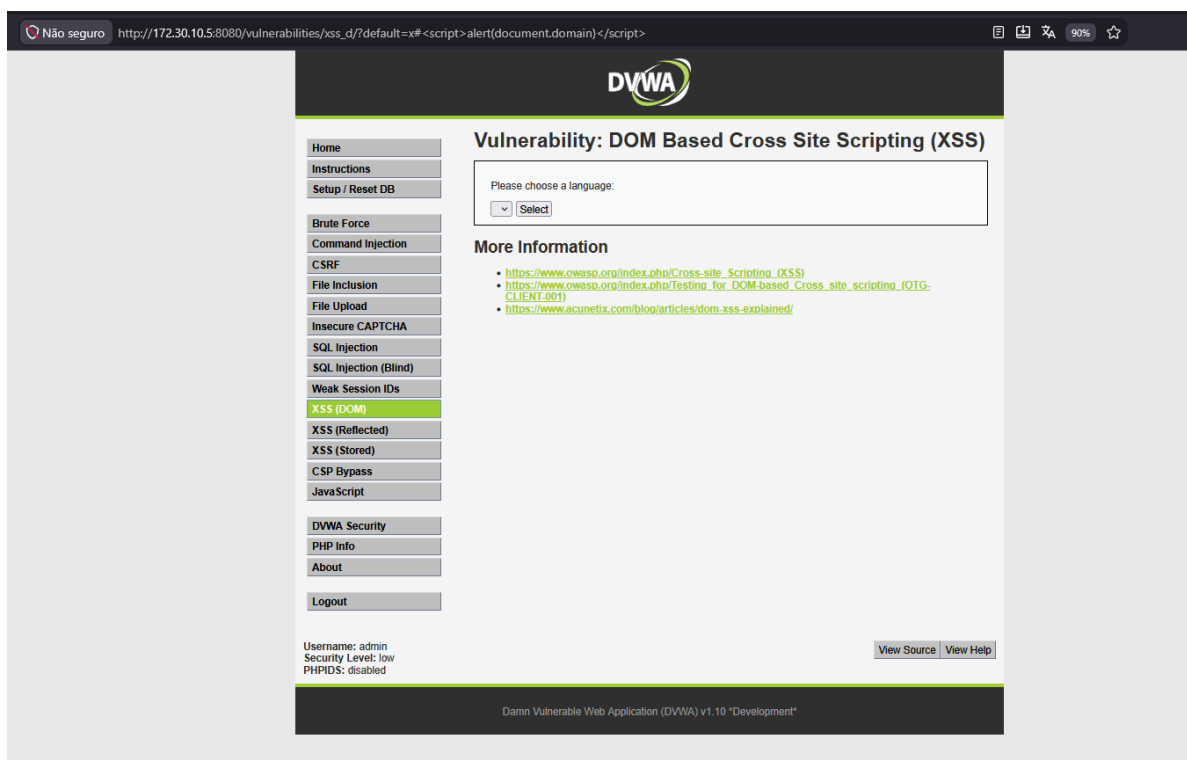


Figura 30 — Correção aplicada: Content-Security-Policy neutraliza o XSS baseado em DOM.

Como recomendação, o Content-Security-Policy estrito bloqueia todo script inline, inclusive o legítimo da aplicação; em um ambiente de produção, o correto é adotar nonces ou hashes por script, permitindo os legítimos e barrando os injetados. A correção definitiva, no entanto, é de aplicação: o código cliente não deve escrever dados não confiáveis diretamente no documento.

7. Contenção e Hardening

Cada elo da cadeia de ataque recebeu uma mitigação, comprovada com a mesma tentativa agora barrada. A abordagem foi de defesa em camadas — corrigindo tanto na rede quanto na aplicação.

7.1. Movimentação lateral (rede)

A regra de firewall que permitia o fluxo DMZ → APP aceitava toda a sub-rede da DMZ como origem — foi o que permitiu ao host comprometido pivotar. A mitigação restringiu a regra a aceitar apenas o WAF como origem legítima. Resultado: o host comprometido (DVWA) passou a receber timeout ao tentar alcançar a API interna, enquanto o caminho legítimo (WAF → API) foi preservado. Uma única regra de rede interrompe a cadeia a partir da movimentação lateral.

7.2. Exposição de credencial e injeção de comando (aplicação)

O endpoint interno que expunha a credencial do banco foi removido, reduzindo a superfície de ataque; após a correção, a requisição retorna 404. A funcionalidade vulnerável a injeção de comando passou a validar a entrada por allowlist e a executar o comando sem shell, com argumentos isolados; após a correção, tentativas de injeção são rejeitadas com 400, enquanto o uso legítimo permanece funcional.

7.3. Redução de superfície e egress

A política default-deny na cadeia FORWARD impede que os segmentos iniciem conexões para fora, contendo reverse shells e exfiltração para a internet. Comprovou-se que nenhum segmento (DMZ ou APP) alcança a internet. Como recomendação adicional, o acesso ao banco deve utilizar um usuário com privilégio mínimo (somente leitura das tabelas necessárias, sem superusuário), reduzindo o impacto de um eventual reúso de credencial.

8. Mapa de Mapeamento de Portas

Nenhuma porta dos serviços de back-end é publicada no host; apenas a VPN e o acesso ao WAF através do túnel são expostos. A tabela consolida o mapa após o hardening.

Host	Porta	Proto	Quem pode acessar	Justificativa
wg-gateway (borda)	51820	UDP	Internet/host	Entrada da VPN — único ponto público
wg-gateway	22	TCP	Sub-rede da VPN	Administração via VPN; login por chave
waf	8080	TCP	Via VPN	Ponto de entrada web; proxy para o DVWA
dvwa	80	TCP	Somente o WAF	Alvo atrás do WAF; nunca direto
app-api	8000	TCP	Somente o WAF	API interna; DMZ não alcança (hardening)
postgres	5432	TCP	Somente a	Banco fechado;

			camada APP	DMZ e internet nunca
--	--	--	------------	----------------------

Comprovação do caminho que deve ser sempre bloqueado: o fluxo DMZ → banco de dados resulta em timeout (recusado), enquanto o fluxo legítimo APP → banco é atendido. O timeout — estado 'filtered' em uma varredura — é a assinatura do firewall descartando o pacote em silêncio, característica de uma segmentação ativa.

9. Resumo dos Achados

#	Vulnerabilidade	Localização	Severidade (CVSS aprox.)
1	SQL Injection	DVWA — módulo SQLi	Crítica (9.8)
2	SQL Injection Blind (time-based)	DVWA — módulo SQLi Blind	Crítica (9.1)
3	XSS Refletido	DVWA — módulo XSS-R	Média (6.1)
4	XSS Armazenado	DVWA — módulo XSS-S	Alta (7.4)
5	XSS baseado em DOM	DVWA — módulo XSS-D	Média (6.1)
6	Upload inseguro → RCE	DVWA — módulo Upload	Crítica (9.8)
7	Command Injection	DVWA — módulo Exec	Crítica (9.8)
8	Local File Inclusion	DVWA — módulo File Inclusion	Alta (7.5)
9	CSRF	DVWA — módulo CSRF	Alta (8.8)
10	Brute Force / Session ID fraco	DVWA — Brute / Weak ID	Alta (7.5)
11	Exposição de credencial + RCE no pivô	API interna	Crítica (9.8)

10. Conclusão, Trade-offs e Escala para a Nuvem

O laboratório demonstrou o ciclo completo — atacar, entender e conter — com evidência em cada etapa. A infraestrutura é reproduzível com um único comando e nenhum segredo é versionado no repositório.

Trade-offs e limitações

- O WAF é um controle importante, porém não suficiente isoladamente: o XSS baseado em DOM via fragmento evidenciou um ponto cego estrutural, mitigado com defesa em profundidade (CSP), e não com uma regra adicional.
- O CSP estrito bloqueia todo script inline; em produção, o correto é o uso de nonces ou hashes por script.
- O alvo é o DVWA, deliberadamente vulnerável, conforme previsto no escopo; as vulnerabilidades da API interna são intencionais e documentadas.

Mapeamento para a infraestrutura real

- Firewall nftables no gateway → Security Groups e NACLs (AWS) ou regras de roteamento no Proxmox; a camada de rede do provedor libera o trânsito até o appliance, que aplica a política fina.
- Segmentos internos → subnets/VPCs separadas por camada, sem rota para a internet, com filtragem de egress explícita.
- WAF Nginx + ModSecurity + CRS → WAF gerenciado (AWS WAF) ou Cloudflare na borda.
- Gateway WireGuard + SSH por chave via VPN → VPN gerenciada ou bastion host isolado.
- Default-deny em INPUT e FORWARD → postura deny-all com liberações explícitas, aplicando menor privilégio à rede.